Improving tilize performance

June 13, 2025

tenstorrent

Agenda

- Why? And current state of tilize
- Initial investigation
- Initial algorithm proposal
- Improved algorithms 1-3
- Integration and pcc issues
- Tilizing arbitrary tensor widths
- Next steps

CONFIDENTIAL - CONTAINS TRADE SECRETS

Why?

- TT-Metal issue #21624.
- Previous effort #17757 (tilize fused with matmul).

  Identified that tilize and matmul are around 2x slower than ideal on tested cases.

  We initially didn't care too much about that as effort was on fusing.

  Once fusing was deemed unperformant with no path forward, logical step was to investigate standalone tilize performance.
- Around the same time Stefan posted conclusions from his tilize benchmarks which further reinforced the need to look into tilize perf.

pavlejosipovic opened on May 5 · edited by pavlejosipovic Edits ···

After moving unet shallow model from from "batched convs" to "grouped convs" at groups =4 conv ops stoped being data movement bound and they became compute bound.
Further investigation of compute kernel performance led to tilize performance being the main bottleneck.

Conv2d is tilizing input0 "on-the-fly" inside compute kernel, and is using `tilize_block` API to perform the tilization.

For largest conv2d in unet model which takes 11% of the whole model 60% of op duration is spend in tilizing the input. Similar pattern of perf distribution holds on all "big" convs in the model.

On WH in theory we have 64 bytes per clock for unpacker bw and given input tile data format is bfloat16 it has 2048 bytes per tile, so in theory we should spends 32 cycle per tile.
In practice we need 75 cycles per tile.
While realising that 32 cycles per tile is just theory 75 cycles per tile is pretty far off and we would benefit from bringing this closer to theoretical bw.

For example brining unpacker bw in range of 40 cycles per tile would get us over the line for target model perf (3k fps on N300, from todays 2450 fps)

Measurements done by LLK team show the same tilize perf, so we are confident that this isn't an op implementation issue.

- Since fusion effort was focused on the case of convolutions, the tilize investigation also primarily looked into that case.
- Standard 32x32 tiles FP16_b input BFP8_b output.

CONFIDENTIAL - CONTAINS TRADE SECRETS

Current state of tilize (on main)

- At the time of initial investigation, a lot of numbers were thrown around.
- While not all numbers where 100% representative and accurate, if was clear that something was off and performance was almost certainly being left on the table.
- 75 clk per tile came from measurements done to compare baseline with fused implementation
  (only few and small block dims).
- While the case of interest was FP16 int BFP8 out, due to poor BFP support in metal C tests, FP16 in FP16 out was often used as a proxy.
  The assumption was that we are unpacker bound and changing the output format wouldn't change the performance FALSE.
- High level numbers from benchmarks (FP16 in/out) were also similar, but don't paint the complete picture until we knew where to look (but that came later).

test	rt_dim	ct_dim	kt_dim	Tilize Per Tile
TilizeMatmul	1	1	1	77.140625
TilizeMatmul	1	1	2	73.7734375
TilizeMatmul	1	1	4	76.28515625
TilizeMatmul	1	2	1	77.234375
TilizeMatmul	1	2	2	73.8671875
TilizeMatmul	1	2	4	75.8046875
TilizeMatmul	1	4	1	76.921875
TilizeMatmul	1	4	2	73.0390625
TilizeMatmul	1	4	4	74.8984375
TilizeMatmul	2	1	1	75.5
TilizeMatmul	2	1	2	73.015625
TilizeMatmul	2	1	4	75.84179688
TilizeMatmul	2	2	1	75.6875
TilizeMatmul	2	2	2	72.82421875
TilizeMatmul	2	2	4	75.43164063
TilizeMatmul	2	4	1	75.546875
TilizeMatmul	2	4	2	72.0078125
TilizeMatmul	2	4	4	74.32617188
TilizeMatmul	4	1	1	74.75
TilizeMatmul	4	1	2	72.6875
TilizeMatmul	4	1	4	75.63964844
TilizeMatmul	4	2	1	74.671875
TilizeMatmul	4	2	2	72.25195313
TilizeMatmul	4	2	4	75.16210938

WH CPP Isolated		
	Estimated unpack tilize	**Measured unpack tilize**
Datatype size[B]	2.00	
Tile width[elements]	32.00	
Tile height[elements]	32.00	
Unpacker BW HW[B/cycle]	64.00	
Packer BW HW[B/cycle]	64.00	
Unpacker BW algo[B/cycle]	64.00	
Packer BW algo[B/cycle]	64.00	
Unpacker [cycles/tile]	32.00	52.51
Packer [cycles/tile]	32.00	44.64
BW Utilization Unpacker	**60.94**	
BW Utilization Packer	**71.68**	

WH CPP OP		
OP Duration [cycles/tile]	**Tilize**	
Data in DRAM	203.613179	
Data in L1	78.16382716	

Avg: 74.76289876

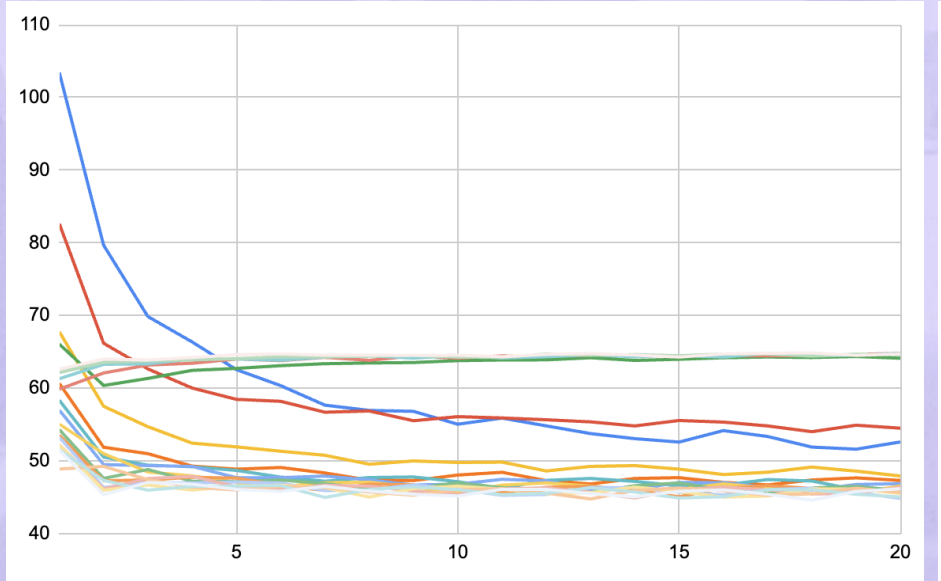
Min: 72.0078125

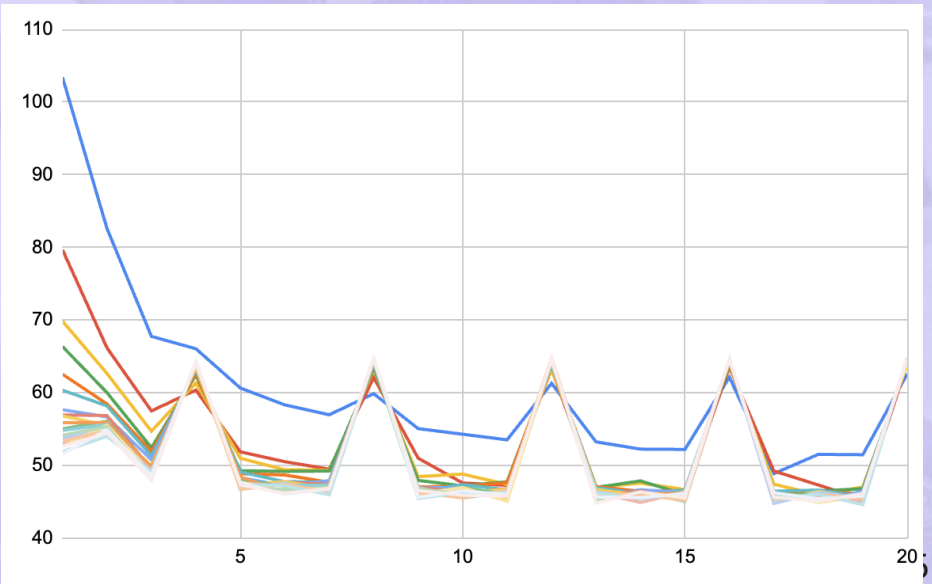
Max: 77.234375

CONFIDENTIAL - CONTAINS TRADE SECRETS

Current state of tilize (on main)

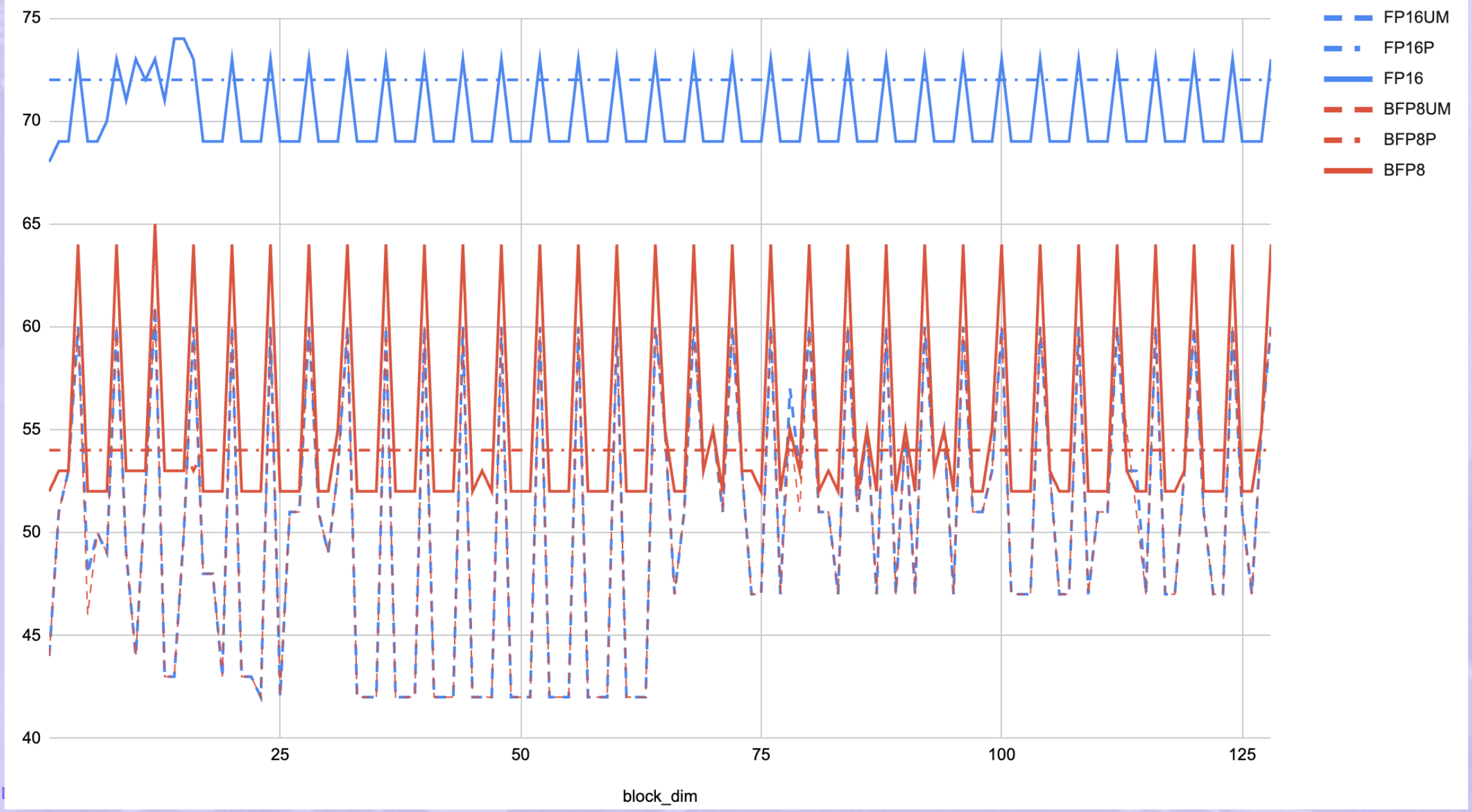
- Reasoning about performance is hard, especially if:
  - You are looking at incomplete data.
  - Starting from a false assumption (complete data helps with this).
  - You are not sure where to look.
- Even if you do everything right, hardware is complex, you can't model everything in your head.
- If you can, best place to look is in waves from the simulation.
- **Hindsight is 20/20** (these conclusions taken from perf data were drawn only after looking at simulations and looking at perf data with that context.





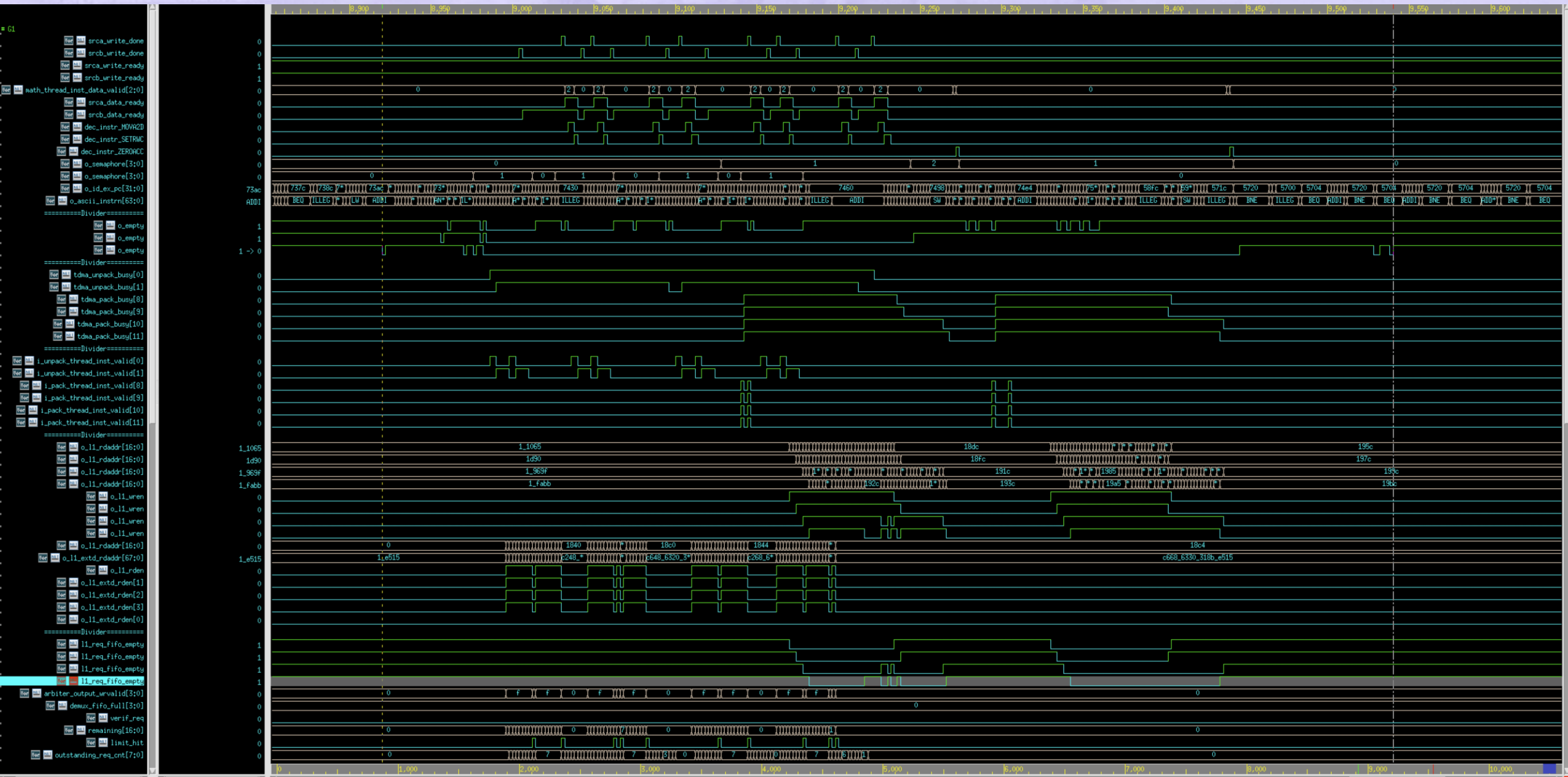
CONFIDENTIAL - CONTAINS TRADE SECRETS

Current state of tilize (on main)



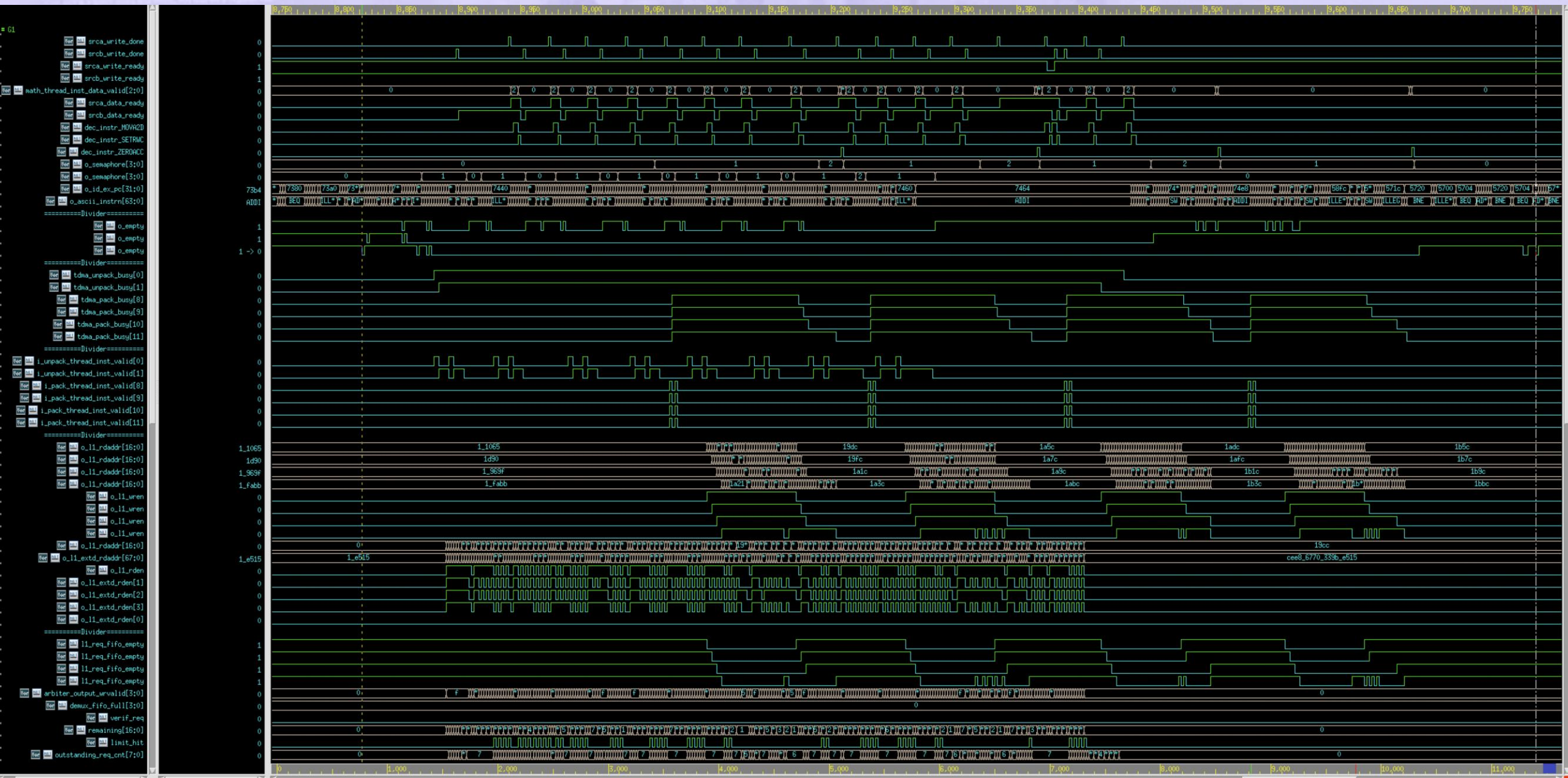
Waves

yyzeon15:/proj_sw/user_dev/lpremovic/wave/wave_whb0_new_tilize.rc



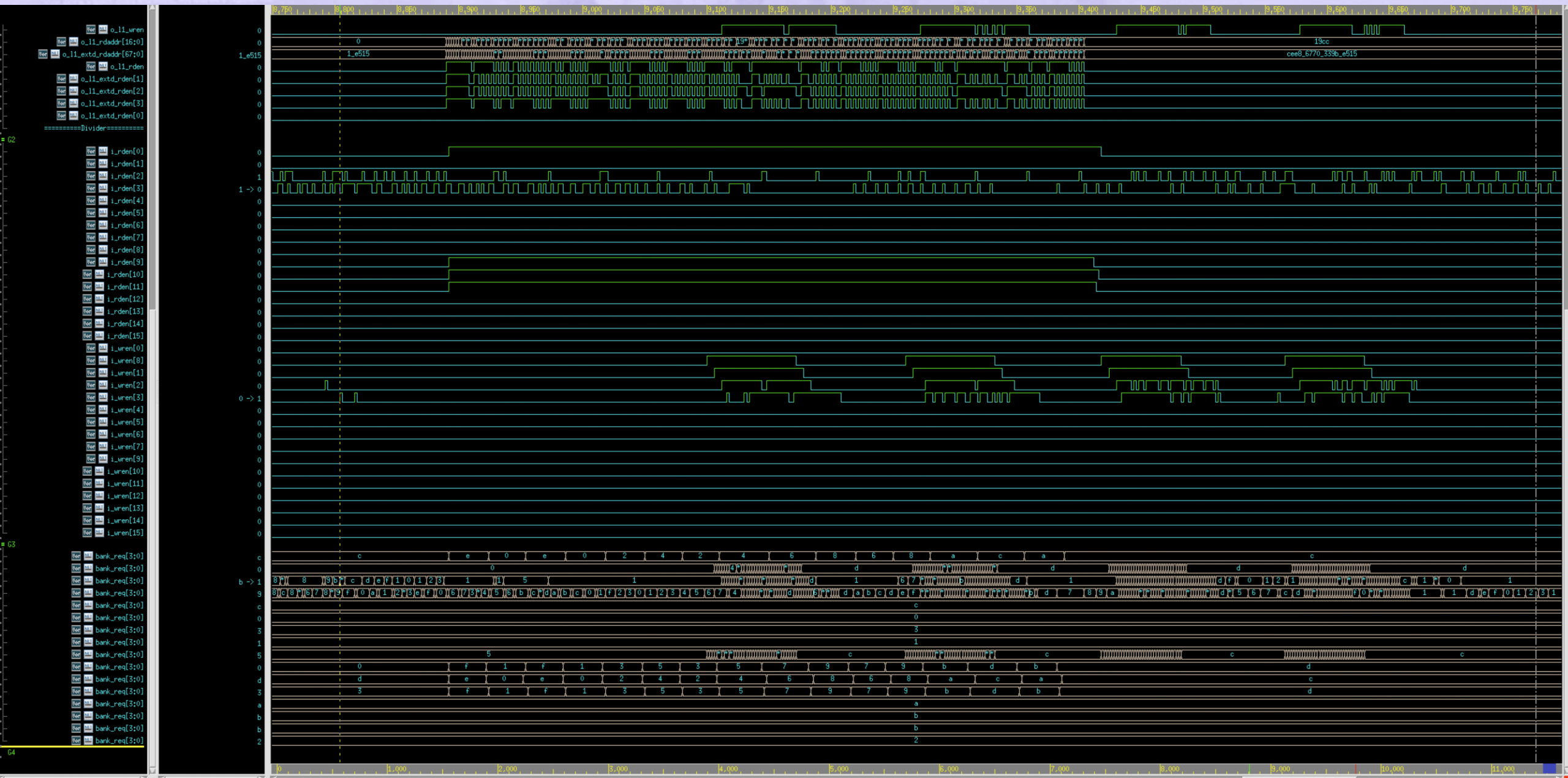
Waves

yyzeon15:/proj_sw/user_dev/lpremovic/wave/wave_whb0_new_tilize.rc



Waves

yyzeon15:/proj_sw/user_dev/lpremovic/wave/wave_whb0_new_tilize.rc



Waves, what to look for

- Src registers: write_done, write_ready, data_ready.
- Math thread: inst_data_valid, dec_instr...
- Semaphores: Math-Pack (1), UnpackContext (5).
- Instruction buffer: empty, full.
- Tdma: unpack_busy, pack_busy.
- Instruction issue: unpack_thread_instrn_valid, pack_thread_instrn_valid.
- Packer: l1_rdaddr, l1_wren.
- Unpacker: l1_rdaddr, l1_rden, l1_extd_rdaddr, l1_extd_rden.
- L1: port rden/wren, bank_req, bank rden/wren.

CONFIDENTIAL - CONTAINS TRADE SECRETS

Initial investigation: Results

- For ~~larger block dims~~ block dims divisible by 4, L1 bank contention.
  Ports 0 and 1 read back to back addreses, then stride for port 2 and ports 2 and 3 again read back to back.
  Ports 0 and 2 and 1 and 3 target the same bank if stride is multiple of 16 words (4 FP16 tiles).

- For all cases, severely risc and pack bound.
  TRISC 0 instruction buffer often goes empty.
  Unpack context semaphore often goes to 0.
  Large gaps in between unpacker L1 activity.
  between adjacent faces (1 cycle) << between top and bottom faces (8 cycles) << between tiles (14 cycles).
  Math Pack semaphore goes to 2.

- Initial focus was on L1 bank contention as that masks risc issues and while doing that tackle some low hanging risc issues.

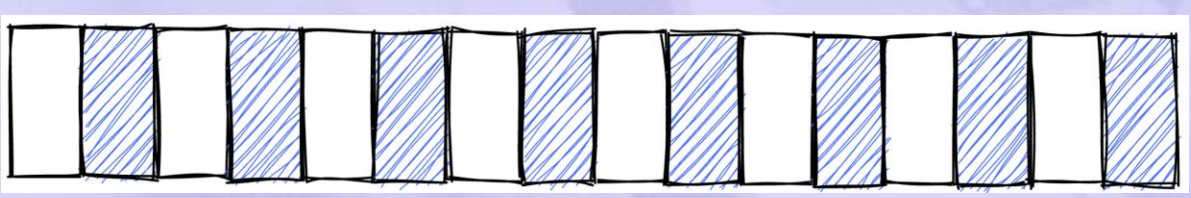
- Changing L1 bank addressing was briefly considered but we weren't sure if there is a one size fits all solution.

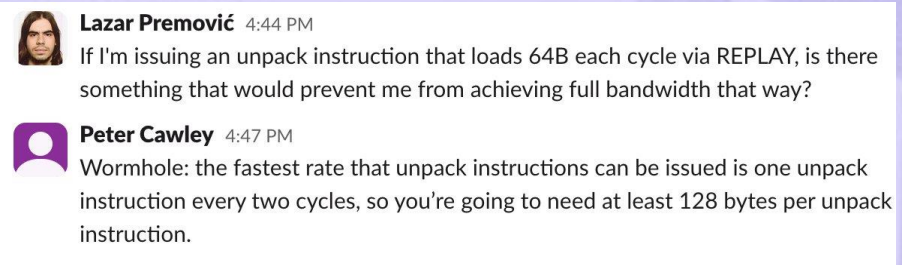
CONFIDENTIAL - CONTAINS TRADE SECRETS

First algorithm

- Make sure that all 4 unpacker interfaces access back to back memory locations in the same cycle, this means loading first row of first two faces before striding for the next cycle.
  This is similar to the behavior on blackhole due to HW bug, but packer is much more flexible here so no perf loss.
- While at it, try to do everything using counters instead of base address manipulation and unpacker contexts.
  Also change unpack to signal data valid once per tile instead of face and try to use as much dest before packing.
- We expected to reach around 60 clk per tile but measured 65.
- On to integration:
  - Optimizing inits
  - Chasing pcc
  - Perf is same or worse e2e :(, what is going on? Back to simulation… 🤦

SRCA:
0 T0F0R0
1 T0F1R0
2 T0F0R1
3 T0F0R1
...
30 T0F0R15
31 T0F1R15
32 T0F2R0
33 T0F3R0
...
62 T0F2R15
63 T0F3R15

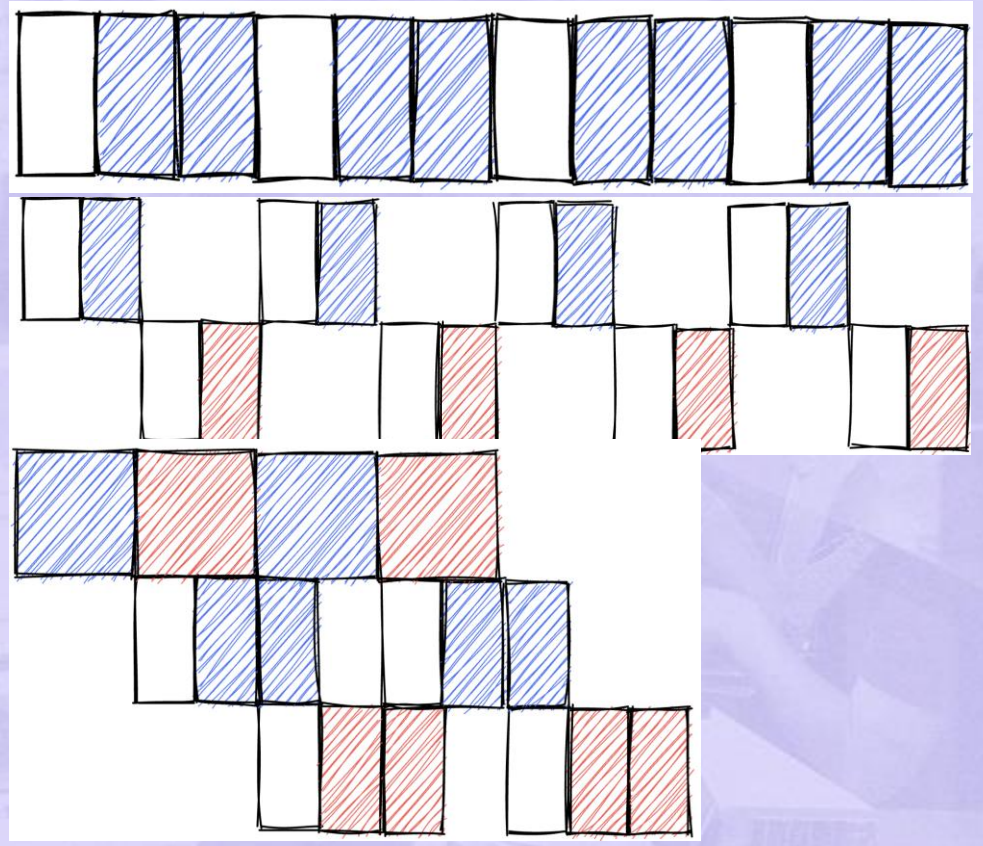




CONFIDENTIAL - CONTAINS TRADE SECRETS

Next algorithms

- Unpack A, 128B reads
- Unpack AB, 64B reads
- Unpack AB 128B reads
  - Chasing packer HW issue



- Since 128B reads mean even number of tiles, need special versions of algorithm for odd number of tiles
  - Block dim == 1 Full dim == 1
  - Block dim == 2 Full dim != 1
  - Block dim == 3 Full dim != 1

SRCA:	SRCA:	SRCA:
0 T0F0R0	0 T0F0R0	0 T0F0R0
1 T0F1R0	1 T0F1R0	1 T0F1R0
2 T1F0R0	2 T0F0R1	2 T1F0R0
3 T1F0R0	3 T0F0R1	3 T1F0R0
0 T0F0R1	...	0 T0F0R1
1 T0F1R1	30 T0F0R15	1 T0F1R1
2 T1F0R1	31 T0F1R15	2 T1F0R1
3 T1F0R1	SRCB	3 T1F0R1
...	0 T0F2R0	...
60 T0F0R15	1 T0F3R0	60 T0F0R15
61 T0F1R15	...	61 T0F1R15
62 T1F0R15	30 T0F2R15	62 T1F0R15
63 T1F1R15	31 T0F3R15	63 T1F1R15
NEXT BANK		SRCB:
0 T0F0R0		0 T0F0R0
1 T0F1R0		1 T0F1R0
2 T1F0R0		2 T1F0R0
3 T1F0R0		3 T1F0R0
0 T0F0R1		0 T0F0R1
1 T0F1R1		1 T0F1R1
2 T1F0R1		2 T1F0R1
3 T1F0R1		3 T1F0R1
...		...
60 T0F0R15		60 T0F0R15
61 T0F1R15		61 T0F1R15
62 T1F0R15		62 T1F0R15
63 T1F1R15		63 T1F1R15

CONFIDENTIAL - CONTAINS TRADE SECRETS

Extras

- https://github.com/tenstorrent/tt-metal/issues/21624
- https://tenstorrent.atlassian.net/browse/TEN-2828
- https://tenstorrent.atlassian.net/browse/TEN-2835
- https://tenstorrent.atlassian.net/browse/TEN-2836
- https://tenstorrent.atlassian.net/browse/TEN-2837

CONFIDENTIAL - CONTAINS TRADE SECRETS

Next steps:

- Removing some of the current limitations:
  - Input operand: 16x16 faces, 4 faces per tile, FP16_b format, any block dim.
  - Output operand: 16x16 faces, 4 faces per tile, FP16_b or BPF8_b format.
  - Dest sync: Half.
  - FP32 dest: False.
  - Goal: Also support FP32 (via TF32) input and FP32 dest.
- Fix edge case issue that surfaces in some integration tests.
- Look into packer perf with HW folks.
- Remove redundant and hardcoded parts of the code.
- Write documentation.
- Merge.

CONFIDENTIAL - CONTAINS TRADE SECRETS